

Claude Multi-Account Fine Tuning

Provider Aliases — User Guide

1. Prerequisites
2. Quick start
3. Commands
 - Useful flags
4. How a provider alias is named and what it runs
5. Config dirs, plugins, and shared state
6. Overrides — `scripts/providers/overrides.json`
`scripts/providers/key-aliases.json`
7. Verification (optional, via LLMsVerifier)
8. Known limitation — default session color
9. Troubleshooting

Provider Aliases — User Guide

`claude-providers` turns every LLM API key you keep in your keys file into its own Claude Code alias, pointed at that provider's strongest model. It reuses the multi-account toolkit's machinery (shared plugins/history, automatic `.bashrc/.zshrc` wiring) so a provider alias behaves like a normal `claudeN` account, just talking to a different model backend.

It is **fully dynamic**: the provider list, base URLs, transports, and model IDs are all derived at run time from your keys file + the [models.dev](#) catalog. Nothing is hardcoded. Re-run it any time to pick up new keys or refresh existing providers.

1. Prerequisites

Need	Why	Install
The toolkit installed	provides <code>lib.sh</code> , the alias file, <code>sync-state</code>	<code>bash scripts/install.sh</code>
A keys file	one <code>export NAME_API_KEY=...</code> per provider	<code>~/api_keys.sh</code> (default) or pass <code>--keys-file</code>
<code>jq</code> , <code>python3</code> , <code>curl</code>	catalog parse + resolve + fetch	system package manager

Need	Why	Install
claude-code-router (only for routed providers)	translate Anthropic [?] OpenAI/ Gemini	npm install -g @musistudio/claude-code-router
submodules/LLMsVerifier built (optional)	full "does this model work?" verification	see §7

Your keys file is the **only** place secrets live. `claude-providers` reads the variable *names* from it (without executing it) to discover providers, and the launch wrapper reads the *value* at the moment you start a session. No key is ever written into the repo, the alias file, or the per-provider env files.

2. Quick start

```
claude-providers sync          # discover + create/refresh all
                             provider aliases
source ~/.local/share/claude-multi-account/aliases.sh # or open
                             a new shell
claude-providers list          # see what you got
deepseek                      # launch Claude Code on DeepSeek's
                             strongest model
```

3. Commands

Command	Does
claude-providers sync (default)	Discover every LLM key, resolve it via models.dev, verify (if available), and create/refresh one alias per provider. Idempotent — safe to run repeatedly.
claude-providers list	Table of installed provider aliases: alias name, provider, transport, strong + fast model.
claude-providers show <id>	Print the resolved env file for one provider.
claude-providers remove <id>	Remove the alias + env file; back up and remove the config dir.
claude-providers add --from-key VAR --id PROVIDER	Register a key→provider mapping (when a key's name doesn't match models.dev), then sync.

Useful flags

- `--keys-file PATH` — use a different keys file (default `~/api_keys.sh`).
- `--no-verify` — skip verification; create aliases regardless.
- `--offline` — don't fetch models.dev; use the local cache (errors if none).
- `--dry-run` — print what would change; write nothing.

4. How a provider alias is named and what it runs

- **Alias name** defaults to the `models.dev` provider id (`deepseek`, `groq`, `mistral`, ...). Want something shorter like `dseek`? Set it in `scripts/providers/overrides.json` (see §6).
- **Strong model** → Claude Code's main model (`ANTHROPIC_MODEL`). Chosen as the provider's most capable: reasoning-capable first, then newest, then largest context.
- **Fast model** → Claude Code's background model (`ANTHROPIC_SMALL_FAST_MODEL`), chosen as the cheapest tool-call-capable model.
- **Transport**:
 - `native` — provider speaks the Anthropic API directly (`models.dev` npm is `@ai-sdk/anthropic`). The alias runs `claude` with `ANTHROPIC_BASE_URL` / `ANTHROPIC_AUTH_TOKEN` / `ANTHROPIC_MODEL` set.
 - `router` — provider is OpenAI-compatible / Gemini. The alias launches through `claude-code-router` (`ccr` code), which translates the protocol.

5. Config dirs, plugins, and shared state

Each provider gets `~/.claude-prov-<id>`, which symlinks the same shared items as your accounts (`projects`, `todos`, `plugins`, `settings.json`, `CLAUDE.md`, ...). So **every plugin you have in `claude1..N` is available in every provider session**, and `history/projects` are unified. `sync` also force-enables the always-on set in shared settings: **superpowers**, **systematic-debugging**, **frontend-design**, **code-review**.

Provider dirs are deliberately **excluded from account auto-detection**, so they never get merged into your real accounts' auth/identity and never interfere with `claude-unify` or `claude-add-account`. Your existing `claudeN` aliases keep working exactly as before, and you can still create new accounts the same way.

6. Overrides — `scripts/providers/overrides.json`

Per-provider pins. Empty by default. Any field you set wins over the `models.dev`-derived value:

```
{
  "deepseek": {
    "alias": "dseek",
    "transport": "native",
    "base_url": "https://api.deepseek.com/anthropic",
    "strong_model": "deepseek-reasoner",
    "fast_model": "deepseek-chat"
  }
}
```

This is how you (for example) promote DeepSeek to its native Anthropic endpoint (`/anthropic`) instead of routing it, or give it the short alias `dseek` — all without touching any code.

scripts/providers/key-aliases.json

Maps a key variable name to a models.dev provider id, for the cases where the name differs (e.g. CODESTRAL_API_KEY → mistral, ZAI_API_KEY → zai). Use `claude-providers add --from-key VAR --id PROVIDER` to append entries.

7. Verification (optional, via LLMsVerifier)

By default sync does a best-effort check (an HTTP probe of the provider's /models, when a key + network are available) and otherwise marks a provider unverified — the alias is **still created** (full verification is opt-in).

For authoritative verification ("does this model exist and can it see my code?") build the LLMsVerifier submodule:

```
git submodule update --init --depth 1 submodules/LLMsVerifier
cd submodules/LLMsVerifier && go build -o bin/model-
verification ./llm-verifier/cmd/model-verification
```

Once submodules/LLMsVerifier/bin/model-verification exists, sync uses it: a provider that **fails** verification is recorded but its alias is **not** activated. (Requires the Go toolchain; the build reads keys from your keys file.)

8. Known limitation — default session color

The original goal was for each provider session to default its /color to **orange**. Investigation of the installed Claude Code (v2.1.178) found that /color is **session-scoped and TUI-only** — it is **never persisted to disk**, and there is no settings key, hook, or environment variable to set a default color. So this cannot be automated with the current Claude Code.

Workaround: type /color orange at the start of a provider session. orange is a valid color. If a future Claude Code adds a persistable color setting, sync will seed it automatically.

9. Troubleshooting

Symptom	Cause / fix
provider 'X' needs claude-code-router	Routed provider, ccr not installed → <code>npm install -g @musistudio/claude-code-router</code> .
\$NAME_API_KEY is empty	The key isn't set in your keys file.
offline and no models.dev cache	Run <code>claude-providers sync</code> once online to populate the cache.
A key didn't become an alias	It's unmapped (no models.dev match) or classified vcs/infra. Map it with <code>add</code> , or check <code>scripts/providers/evidence/mapping-report.md</code> .
Alias missing after sync	

Symptom	Cause / fix
	<code>source ~/.local/share/claude-multi-account/aliases.sh</code> or open a new shell.